

REFACTORING DOCUMENT

POTENTIAL REFACTORING TARGETS

In order to identify potential refactoring targets we have chosen portions of the codebase which we have deemed to be the most important in the sense that it has a major role in the functioning of our project.

List of potential refactoring targets:

GameEngine start_game(),
Player issue_order(),
GameEngine getNumOfReinforcements(),
MapEditor readMap(),
Order execute(),
Player class structure,
WarMap class structure,
WarMap showMap(),
WarMap showMap(List of Players),
WarMap validateMap(),
MapEditor editMap(),
MainGameLoop run_game_loop(),
MapEditor editMapEntry(),
Player next_order(),
WarMap edit functions.

ACTUAL REFACTORING TARGETS

In order to choose the following refactoring targets, we have chosen functions which by refactoring allowed us to most easily implement the requirements of build 2 as some of these functions would not allow us to easily implement the requirements given prior to the refactoring.

List of actual refactoring targets:

GameEngine start_game(),
Player issue_order (),
Order execute(),
MainGameLoop run_game_loop(),
MapEditor editMapEntry()

Refactoring Operations

Refactoring operation #1 (GameEngine start_game())

The tests that apply to this class include:

testAssignCountries,
testAddPlayer,
testRemovePlayer,
testGetNumOfReinforcements,
testEditPhaseTransition,
testPlayPhaseTransition,
testStartUpValidation.

The refactoring operation was deemed necessary as the previous implementation of GameEngines play function could not easily implement the State pattern, therefore we refactored this function in order to be able to more easily implement the state pattern.

Prior to the refactoring the GameEngine play function only retained control of the program during what would now be considered the start up phase, and would no longer get commands once the start up phase was completed. With the refactoring the GameEngine play function hold's control of the program when receiving commands, and all the different phases occur within the play function rather than control being passed to either the MapEditor or MainGameLoop as was done prior to the refactoring.

Refactoring operation #2 (Player issue_order ())

The tests that apply to this class include:

testNextOrderEmptyList,
testNextOrder,
testInvalidCountry,
testCannotDeployMoreArmiesThanReinforcements,
testBlockadeIssueOrderValidInput,
testBlockadeIssueOrderInvalidCountry,
testBlockadeIssueOrderNoBlockadeCard,
testBlockadeCommandExecution,
testAdvanceCommandExecution,
testAdvanceCommandExecutionWithInvalidSourceCountry,
testBombCommandExecution,
testBombCommandExecutionWithInvalidCountry,
testBombCommandExecutionWithoutBombCard,
testDiplomacyCommandExecution,
testDiplomacyCommandExecutionWithoutDiplomacyCard,
testDiplomacyCommandExecutionWithInvalidTargetPlayer,
testAirliftCommandExecution,
testAirliftCommandExecutionWithInvalidSourceCountry,

The refactoring operation was deemed necessary as the previous implementation of Player's issue_order function could not easily implement the command pattern, furthermore it could not be used in the State pattern as it retained control of the program for what would now be considered the entire issue order phase.

Prior to the refactoring operation the issue_order function would retain control of the program until all players have issued all of their orders, it is now called repeatedly and issues a single order at a time based on the input, which takes the input command from

the player in order to create a singular order of the appropriate order type making use of the command pattern.

Refactoring operation #3 (Order execute()):

The tests that apply to this class include:

testExecute,
testExecuteBlockadeOrderNonExistingCountry,
testExecuteBlockadeOrderCountryRemoved,
testExecuteBlockadeOrderNumOfArmies,
testBlockadeCommandExecution,
testAdvanceCommandExecution,
testAdvanceCommandExecutionWithInvalidSourceCountry,
testBombCommandExecution,
testBombCommandExecutionWithInvalidCountry,
testBombCommandExecutionWithoutBombCard,
testDiplomacyCommandExecution,
testDiplomacyCommandExecutionWithoutDiplomacyCard,
testDiplomacyCommandExecutionWithInvalidTargetPlayer,
testAirliftCommandExecution,
testAirliftCommandExecutionWithInvalidSourceCountry

The refactoring operation was deemed necessary as the previous implementation did not make use of the command pattern as it assumed all order's were deploy orders. We therefore needed to refactor it to be used by any type of order.

Prior to the refactoring it was only applicable to the deploy order, now it makes use of the command pattern in order to run the appropriate execute function from the implemented order types, this also meant creating additional execute functions for each order type and having Order execute call those depending on the type of order.

Refactoring operation #4 (MainGameLoop run_game_loop())

The tests that apply to this class include:

testGetNumOfReinforcements,
testPlayPhaseTransition,
testStartUpValidations,
testAssignCountries

The refactoring operation was deemed necessary as the previous implementation retained control of the program for all of the main game loop, which was detrimental to using the state pattern.

Prior to the refactoring the MainGameLoop's play function would retain control of the program for the entirety of the main game play, this was refactored to have its functionality implemented in the Play phases in order to make use of the state pattern, furthermore only one game play command is executed before returning control back to the GameEngine instead of the MainGameLoop holding control for the entire time.

Refactoring operation #5 (MapEditor editMapEntry())

The tests that apply to this class include:

testIsContinentConnected,
testIsConnectedGraph,
testIsEmptyContinent,
testingisCountryNoNeighbor,
testEditPhaseTransition

The refactoring operation was deemed necessary as the previous implementation retained control of the program for all map editing, which was detrimental to using the state pattern.

Prior to the refactoring the map editor would run this function in what was effectively a loop in order to edit a map completely. After the refactoring all functionality was moved to the Edit phases to make use of the state pattern, furthermore only one edit operation is executed before returning control to the GameEngine instead of the map editor holding control the entire time.